Text-to-Wikidata Entity Linking: a GraphRAG-inspired Pipeline

Goal: a lightweight Python pipeline that identifies meaningful terms (not only named entities) in unstructured text and links each to the right Wikidata concept. **Hypothesis:** graph context from Wikidata improves the entity-linking (disambiguation) step. The pipeline runs fully locally (spaCy + small Ollama models), and every design decision below was made based on the experiments done on the candidate generator and the ranker. *The appendix and references are outside the 2-page limit.*

Dataset: built from ELEVANT [1] (EMNLP 2022, Uni Freiburg), chosen because it annotates entities beyond named entities. Gold QIDs were re-validated against current Wikidata (the dataset is old), and the set was stratified into ambiguous vs non-ambiguous mentions: a surface form mapping to more than one gold QID across the dataset is flagged ambiguous, since the mention string alone cannot disambiguate it. The stratification also uses the NER outcome of step 1 — each mention is labelled ambiguity × NER-extracted (ambiguous/non-ambiguous × ner/no-ner), so the evaluation can separate what gets linked when NER already found the mention from when it did not. Documents were then sampled by taking the documents with the most mentions, keeping all mentions of a chosen document and breaking ties to keep the ambiguous/non-ambiguous split closest to even — multi-sentence paragraphs with several mentions, so that coherence between mentions can actually act.

1. Text processing and term extraction

- spaCy (small model) splits paragraphs into sentences and extracts POS, noun phrases, lemmas and named entities. Each mention is expanded with its lemma, stem and WordNet synonyms for retrieval.
- **Decision 1 — spaCy alone misses non-named-entity mentions.** To keep the solution lightweight, instead of a large NER model I added d1-omni:600m (Ollama), a 600 M-parameter decision model: an encoder that takes a state (the text) plus typed questions with defined NER types and returns a calibrated probability per option in one forward pass, with no token-by-token generation. Merging spaCy with the decision model raised mention coverage from 61/94 (~65%, spaCy alone) to 90/100.

2. Candidate retrieval (Wikidata API)

- Each mention form (surface, lemma, stem, synonyms) is queried in parallel (multiprocessing); candidates are cached in a SQLite store with QID, label, description, aliases and 1-hop triples. This stage is tuned for recall — the gold entity should be among the candidates; selection is left to the ranker.
- **Decision 2 — wbsearchentities over CirrusSearch.** Two APIs were compared: wbsearchentities and list=search (Cirrus full-text with sentence context). Cirrus underperformed: less prominent entities that merely contain the query words surface first, causing a large recall drop. wbsearchentities ranks by prominence (sitelinks / incoming links), which matches how mentions are used in text, so it was chosen.
- Candidate recall: 68/90 = 75.6% overall; 41/46 = 89.1% on non-ambiguous and 27/44 = 61.4% on ambiguous mentions.
- Negative result behind Decision 2: with Cirrus full-text search instead, candidate recall drops to 35/90 = 38.9% (non-ambiguous 16/46 = 34.8%, ambiguous 19/44 = 43.2%).
- **Insight.** Candidate recall is the ceiling on end-to-end accuracy, and the misses concentrate on ambiguous mentions (61.4% vs 89.1%) — on those, the generator, not the ranker, is the bottleneck. Recall-oriented query expansion (lemma, stem, synonyms) plus prominence-ranked search are what hold overall recall at 75.6%.

3. Knowledge graph enrichment

- For every candidate, its Wikidata triples (properties and related entities) are fetched and stored. Per document, a NetworkX graph connects all candidates of all mentions: nodes are QIDs, edges are Wikidata properties, and both keep their RDF terms (wd:/wdt: IRIs), following the RDF/SKOS framing.
- This graph is the disambiguation signal: documents talk about related things, so correct candidates pull together into a connected cluster — e.g. Belarus, Germany and Ireland link directly via “diplomatic relation” — while wrong candidates (the element germanium, the person Samuel Ireland) stay isolated at the edges (figure in Appendix, Fig. A1).

4. GraphRAG-based candidate selection — ranker architecture and experiments

Base scorer (all variants): cosine similarity between the embedding of “mention + sentence” and each candidate rendered as “label: description (aliases)” (embeddinggemma), plus small lexical priors: Wikidata's own search rank (0.15/rank), exact label match (+0.10) and a Levenshtein penalty (−0.05). GraphRAG enters in two ways:

- **Retrieval-augmentation:** triples are retrieved at candidate-generation time, and each candidate's graph neighbourhood is verbalised into up to 12 facts (“diplomatic relation Germany; …”), with facts naming the sentence or other mentions' candidates ranked first, and appended to the candidate text before embedding.
- **Graph coherence bonus (weight 0.2):** a candidate directly connected to another mention's candidate scores 1.0, a shared neighbour 0.5, an isolated candidate 0 — the right answers are the ones that fit together.

Seven ranker variants (cr_v1–v7) were built on this scorer and compared in two stages: a pilot on the ambiguous rows chose the directions to pursue, and the full stratified run chose the final ranker. All seven results, including the negative ones:

Ranker	Pilot (ambig.) acc/MRR	Overall acc/MRR	Ambiguous	Non-ambiguous	Time
v1 embed (embeddinggemma)	50.0% / 0.53	57.8% / 0.64	40.9% / 0.48	73.9% / 0.79	16 s
v2 embed + graph	50.0% / 0.55	63.3% / 0.67	47.7% / 0.52	78.3% / 0.82	23 s
v3 d1 (d1-omni:600m)	54.5% / 0.59	–	–	–	4 s
v4 d1 + graph	59.1% / 0.61	–	–	–	4 s
v5 AMR graph (amrlib)	54.5% / 0.57	–	–	–	1571 s

Ranker	Pilot (ambig.) acc/MRR	Overall acc/MRR	Ambiguous	Non-ambiguous	Time
v6 AMR + d1	45.5% / 0.53	–	–	–	11 s
v7 graph-embed (NetMF-style)	–	57.8% / 0.64	38.6% / 0.47	76.1% / 0.80	19 s

Pilot = ambiguous rows only; "–" = not run at that stage. Times: full stratified run for v1/v2/v7, pilot for v3–v6 (v1/v2 pilot times were 5–6 s).

Decisions made from these experiments:

- **Embedding vs decision-model ranking.** v3/v4 phrase ranking as one d1 choice question (candidates as options, calibrated probability per candidate as the score). On the pilot, d1 + graph (v4, 59.1%) beat the embedding ranker, and in both families adding graph context helped (v1→v2, v3→v4) — supporting the GraphRAG hypothesis independently of the ranking model.
- **Verbalised vs structural graph embeddings.** v7 replaces the verbalised facts with structural node embeddings (NetMF-style: degree-normalised random-walk matrix, log-damped, SVD-factorised) and scores each candidate by cosine to the centroid of the other mentions' candidate vectors, deliberately damping the hub-popularity bias of the verbalised bonus. It matched the no-graph baseline overall (57.8%) but lost to verbalised graph context on ambiguous mentions (38.6% vs 47.7%).
- **Negative results — AMR graphs (v5/v6).** v5 parses each sentence into an AMR graph (amrlib) and matches graph-to-graph against the candidates' triples; it edged past the embedding baseline on the pilot (54.5% vs 50.0%) but took 1571 s — two orders of magnitude slower — ruling it out. v6 feeds the same AMR facts into a d1 choice question and was the worst variant (45.5%): stacking verbalised graphs on both sides dilutes the evidence instead of sharpening it.
- **Final ranker: v2 (embeddings + verbalised graph facts + coherence bonus).** Graph context lifts overall accuracy 57.8% → 63.3% and ambiguous accuracy 40.9% → 47.7% over the no-graph baseline. The top candidate is returned only when evidence is sufficient (score/graph support), otherwise alternatives are surfaced.
- **Insight.** Graph context is the one change that helps in both ranking families (v1→v2, v3→v4), and its gain lands exactly where disambiguation is hardest — ambiguous mentions (+6.8 points). How the graph is used matters more than the ranking model: verbalised facts beat both structural embeddings (v7) and AMR graphs (v5/v6). MRR sitting above accuracy (0.67 vs 63.3%) means that when rank 1 is wrong the gold entity is usually just below it, so the surfaced alternatives often contain the right answer.

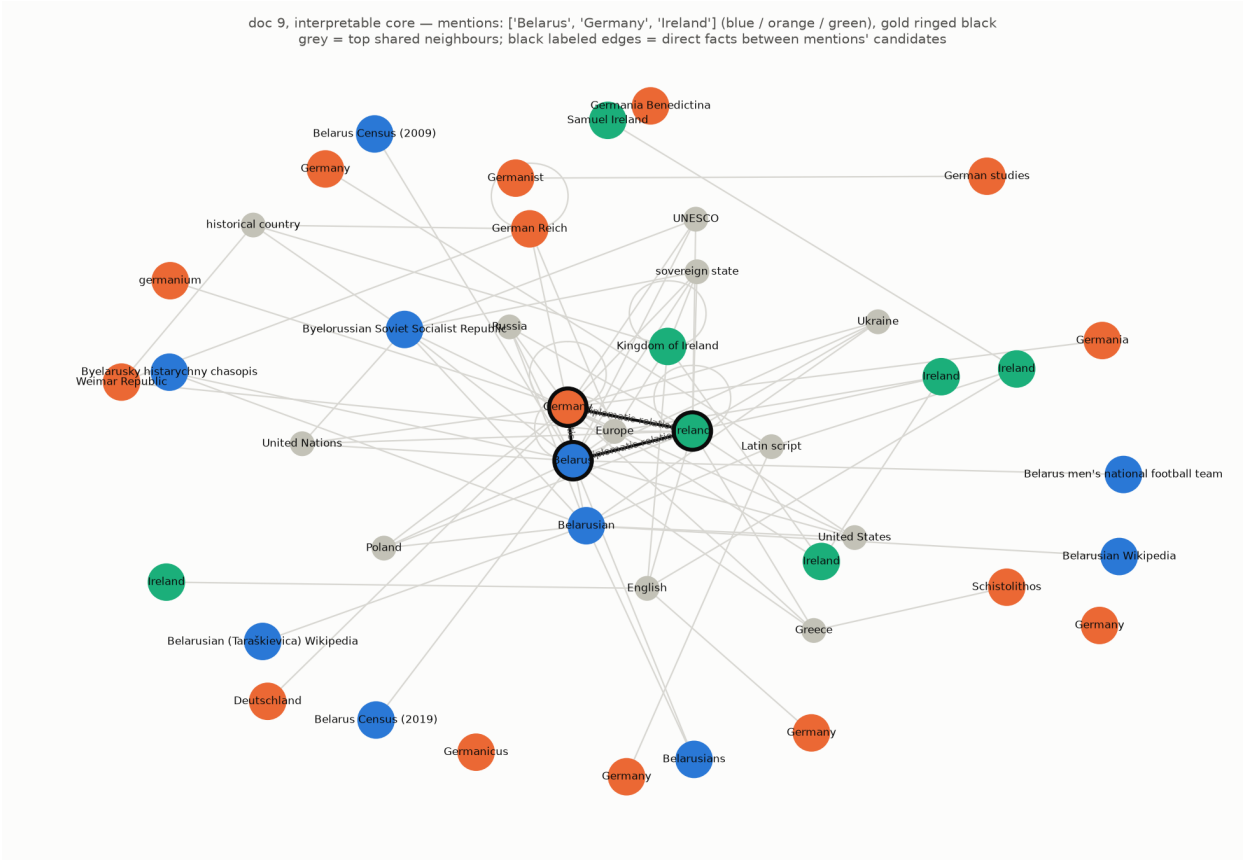
5. Output and evaluation

- Output is structured JSON per mention: original term and sentence, selected concept with QID, rank, score, confidence and supporting evidence (the matched description and graph facts), plus the ranked alternative candidates.
- Evaluation compares the no-graph baseline against the graph-context ranker on the stratified set (table above): the graph context gives the largest gain exactly where disambiguation is hardest, confirming the hypothesis.

AI usage declaration. AI assistance was used for code polishing and for paraphrasing the write-up. The dataset selection, the candidate-generator and ranker experiment design, the decision and evaluation criteria, and the baseline code implementation are the author's own.

Appendix

Fig. A1 — Document-level candidate graph (document 9). The correct candidates (Belarus, Germany, Ireland) form a tightly connected core via "diplomatic relation" edges, while wrong candidates with the same surface forms (the chemical element germanium, the person Samuel Ireland) have no connections to the rest of the document and drift to the periphery. The coherence bonus scores exactly this structure.



References

[1] Hannah Bast, Matthias Hertel, Natalie Prange. 2022. ELEVANT: A Fully Automatic Fine-Grained Entity Linking Evaluation and Analysis Tool. In Proceedings of EMNLP 2022: System Demonstrations. https://aclanthology.org/2022.emnlp-demos.8/